



boot.img 和 resource.img

1.1 概述

RK 平台支持 3 种类型的 kernel 固件（也就是 boot.img 镜像）：RK 格式（RK 自己定义的吧）、Android 格式、Distro 通用格式。

说明如下：

- RK 格式

镜像文件的 magic 为“KRNL”：

```
00000000  4B 52 4E 4C 42 97 0F 00 1F 8B 08 00 00 00 00 00
KRNL..y.....
00000010  00 03 A4 BC 0B 78 53 55 D6 37 BE 4F 4E D2 A4 69
.....xSU.7.ON..i
```

kernel.img = kernel;

resource.img = dtb + logo.bmp + logo_kernel.bmp;

boot.img = ramdisk;

recovery.img = ramdisk(for recovery) ;

- Android 格式

镜像文件的 magic 为“ANDROID!”：

```
00000000  41 4E 44 52 4F 49 44 21 24 10 74 00 00 80 40 60
ANDROID!$.t...@`
00000010  F9 31 CD 00 00 00 00 62 00 00 00 00 00 00 F0 60
.1.....b.....`
```

boot.img = kernel + ramdisk+ resource + <dtb>;

recovery.img = kernel + ramdisk(for recovery) + resource + <recovery_dtbo> + <dtb>;

说明：recovery_dtbo：从Android-9.0才开始新增的镜像；dtb：从Android-10.0才开始新增的镜像；

- Distro格式

目前开源 Linux 的一种通用固件打包格式: 将 ramdisk、dtb、kernel 打包成一个镜像。这个镜像通常以某种文件系统格式存在, 例如 ext2、ext4、fat 等, U-Boot需要通过文件系统才能访问到它的内容。更多参考:

```
./doc/README.distro
./include/config_distro_defaults.h
./include/config_distro_bootcmd.h
```

- 引导优先级: android > rk > distro, 每一类固件都有对应的启动命令, 三个命令会按优先级逐个执行, 直到把固件引导起来。如果所有命令都失败, 则停在U-Boot命令行模式。

启动优先级定义:

```
#define RKIMG_BOOTCOMMAND \
    "boot_android ${devtype} ${devnum};" \
    "bootrkp;" \
    "run distro_bootcmd;"
```

U-Boot 中定义了这三种镜像的启动流程:

```
91
92 #undef RKIMG_BOOTCOMMAND
93 #define RKIMG_BOOTCOMMAND \
94     "boot_fit;" \
95     "boot_android ${devtype} ${devnum};" \
96     "run distro_bootcmd;"
97 #endif
```

这个跟前面那个图中不一样, 因为前面那个图是 RK 文档中的截图, 可能是不同的 SoC 的镜像吧。

1.2 resource.img

resource.img 是 RK 自己定义的一个镜像文件, 而并非是 Android 原本就有的。

其实这个镜像文件包含了一些 logo 图片 (logo.bmp logo_kernel.bmp)、还有内核设备树 dtb 镜像 logo.bmp logo_kernel.bmp 这两个 bmp 图片都在内核的顶层目录下, 如下图所示:

build.config.gki_kprobes	CREDITS	Kbuild	Makefile	samples	vmlinux
build.config.gki_kprobes.aarch64	crypto	Kconfig	mm	scripts	vmlinux.o
build.config.gki_kprobes.x86_64	Documentation	kernel	modules.builtin	security	zboot.img
build.config.gki.x86_64	drivers	kernel.img	modules.order	sound	
build.config.x86_64	firmware	lib	Module.symvers	System.map	
built-in.a	fs	LICENSES	net	tests	
certs	include	logo.bmp	OWNERS	tools	
configs	init	logo_kernel.bmp	README	usr	
COPYING	ipc	MAINTAINERS	resource.img	virt	

内核编译的过程中会打印出 resource.img 镜像包含的文件:

```
found ./arch/arm64/boot/dts/rockchip/.rk3568-evb1-ddr4-v10.dtb.dts.tmp
Image: resource.img (with rk3568-evb1-ddr4-v10.dtb logo.bmp logo_kernel.bmp) is ready
Image: boot.img (with Image resource.img) is ready
Image: zboot.img (with Image.lz4 resource.img) is ready
```

RK 提供了一个工具用于生成 resource.img 镜像, 这个工具在内核源码目录下 scripts/resource_tool, 这不是一个脚本, 而是一个可执行的二进制文件, 通过源码 scripts/resource_tool.c 编译而成。通过下面这条命令就直接生成了 resource.img:

```
scripts/resource_tool ${DTB_PATH} ${LOGO} ${LOGO_KERNEL} >/dev/null
```

具体是如何生成的 resource.img 的, 这个需要查看 scripts/resource_tool.c 源码的内容了。

RK 的内核在编译完之后会帮我们自动生成 resource.img (直接在内核源码顶层目录下), 其实这个 scripts/resource_tool 是在 scripts/mkimng 工具中被调用的。这个工具也是 RK 自己写的。

从上图我们可以知道, 此时 resource.img 镜像中只包含了内核设备树 dtb、以及两个 logo 文件, 但事实上, RK 提供的 resource.img 镜像中还有其它几个图像资源文件, 也就是充电时显示的几张图片资源, battery_0.bmp、battery_1.bmp、battery_2.bmp、battery_3.bmp、battery_4.bmp、battery_5.bmp、battery_fail.bmp, 但是这几张图片并不在内核源码目录中, 而是在 u-boot 源码目录中, 所以当我们直接在内核源码编译时得到的 resource.img 镜像中并不包含 u-boot 源码中的这些充电相关的图片资源。

那充电相关的图片资源是在哪里包含的呢? 而是在安卓顶层目录下的 build.sh 脚本中(RK 写的脚本), 在 build.sh 脚本中有这样一条指令:

```
echo "package resource.img with charger images"
cd u-boot && ./scripts/pack_resource.sh ../kernel/resource.img && cp resource.img ../kernel/resource.img && cd -
```

在执行这条命令之前, 如果我们调用 “./build.sh -UKA”, 前面已经编译了内核源码和 u-boot 源码, 而编译安卓是在这条命令之后。从这条命令可知, 它会先进入到 u-boot 目录下, 然后通过 ./scripts/pack_resource.sh 脚本来制作一个新的 resource.img 镜像。而命令后面指定的参数 ../kernel/resource.img, 就是编译内核源码时生成的 resource.img 镜像文件, pack_resource.sh 脚本会在这个镜像的基础上将充电相关的图片添加进去, 这些图片资源在 u-boot 源码目录下的 tools/images 中, 所以脚本会扫描出该目录中的所有文件, 将其添加到 resource.img 镜像中。原理就是先将内核目录下的 resource.img 镜像解包, 之后再打包一次即可! 得到新的 resource.img 镜像 (在 u-boot 顶层目录下) 之后将其拷贝到内核源码目录下替换旧的 resource.img。

对于 resource.img 镜像文件, 我们可以使用 u-boot 目录下的 scripts/unpack_resource.sh 脚本对其进行解包! 以获取到该镜像中所包含的文件。

1.3 boot.img

从上面可知, 我们直接在编译内核源码时, 除了会生成 resource.img 镜像之后, 还会生成 boot.img 镜像, 并且由上面的图可知, boot.img 镜像是由 Image (内核镜像) 和 resource.img 镜像所构成的。打印信息出处 (scripts/mkimng), 在 mkimng 中有这样一个函数:

```

make_boot_img()
{
    RAMDISK_IMG_PATH=${objtree}/ramdisk.img
    [ -f ${RAMDISK_IMG_PATH} ] && RAMDISK_IMG=ramdisk.img && RAMDISK_ARG="--ramdisk ${RAMDISK_IMG_PATH}"

    ${srctree}/scripts/mkbootimg \
        ${KERNEL_IMAGE_ARG} \
        ${RAMDISK_ARG} \
        --second resource.img \
        -o boot.img && \
    echo " Image: boot.img (with Image ${RAMDISK_IMG} resource.img) is ready";
    ${srctree}/scripts/mkbootimg \
        ${KERNEL_ZIMAGE_ARG} \
        ${RAMDISK_ARG} \
        --second resource.img \
        -o zboot.img && \
    echo " Image: zboot.img (with ${ZIMAGE} ${RAMDISK_IMG} resource.img) is ready"
}

```

该函数用于生成 boot.img 镜像，其实就是通过 scripts/mkbootimg 程序来生成的，具体用法可以使用--help 来查看帮助信息。这个 mkbootimg 是安卓官方提供的一个 Python 脚本，用于制作安卓格式的 boot.img 镜像。从命令中可知，\${KERNEL_ZIMAGE_ARG} 就是内核镜像、RAMDISK_ARG 就是 ramdisk.img 镜像，resource.img 就是内核编译生成的 resource.img 镜像。

并且通过打印可知，其实 RAMDISK_ARG 是空的，所以才会导致上面的打印信息中，boot.img 只包含了内核镜像和 resource.img 镜像。因为 ramdisk.img 是由安卓生成的，并不是编译内核源码生成的。

事实上，编译内核源码所得到的 boot.img 镜像其实是没用的（不会使用它生成的 boot.img 镜像），首先没包含 ramdisk.img，其次 resource.img 镜像还少了一些图片资源，最后还有 cmdline 没包含在其中。我们在安卓顶层调用“build.sh -KUA”去编译安卓的时候，其实会再次去生成 boot.img 镜像文件，也就是在 out 目录下生成的 boot.img 镜像才是最终的 boot.img 镜像，这是如何生成的呢？

build.sh 脚本最终会去执行 build/make/core/Makefile，在这个文件中有这样的命令：

```

$(INSTALLED_BOOTIMAGE_TARGET): $(MKBOOTIMG) $(INTERNAL_BOOTIMAGE_FILES)
    $(call pretty,"Target boot image: $@")
    $(hide) $(MKBOOTIMG) $(INTERNAL_BOOTIMAGE_ARGS) $(INTERNAL_MKBOOTIMG_VERSION_ARGS) $(BOARD_MKBOOTIMG_ARGS) --output $@
    $(hide) $(call assert-max-image-size,$@,$(BOARD_BOOTIMAGE_PARTITION_SIZE))

```

其实就是调用了这个：

```
$(MKBOOTIMG) $(INTERNAL_BOOTIMAGE_ARGS) $(INTERNAL_MKBOOTIMG_VERSION_ARGS) $(BOARD_MKBOOTIMG_ARGS) --output $@
```

生成的 boot.img 镜像文件。也就是使用了\$(MKBOOTIMG)工具，通过打印信息可知，其实就是 ./out/host/linux-x86/bin/mkbootimg 程序，out 目录是编译所生成的目录，所以可知 out/host/linux-x86/bin/mkbootimg 也是在编译过程中所生成的二进制文件，和内核源码目录下的 scripts/mkbootimg 有点不一样，但是它们的使用方法却是相同（除此之外，我还发现 u-boot 目录下也有 ./u-boot/scripts/mkbootimg，文件大小也不相同 不知道为啥）。

先不管这些了，事实上 out/host/linux-x86/bin/mkbootimg 其实就是安卓顶层目录下的 system/tools/mkbootimg/mkbootimg.py 这样的一个 Python 脚本变成的一个可执行文件，当然你也可以直接使用这个 mkbootimg.py，此外在 system/tools/mkbootimg 目录下还有解包脚本 unpack_bootimg.py。

知道安卓制作 boot.img 的工具之后，我们再来看看：

```
$(MKBOOTIMG) $(INTERNAL_BOOTIMAGE_ARGS) $(INTERNAL_MKBOOTIMG_VERSION_ARGS) $(BOARD_MKBOOTIMG_ARGS) --output $@
```

打印了参数列表，如下所示：

```

out/host/linux-x86/bin/mkbootimg
--kernel out/target/product/rk3568_r/kernel --ramdisk out/target/product/rk3568_r/ramdisk.img --dtb out/target/product/rk3568_r/dtb.img --cmdline console=ttyFIQ0 androidboot.baseband=
M/A androidboot.wificountrycode=CN androidboot.verifymode=enforcing androidboot.hardware=rk38board androidboot.console=ttyFIQ0 androidboot.verifiedbootstate=orange firmware_class.pat
h=/vendor/etc/firmware init/init rootwait ro loop.max_part=7 androidboot.selinux=permissive buildvariant=userdebug
--os version 11 --os patch level 2021-06-05
--second kernel/resource.img --header version 2

```

--kernel out/target/product/rk3568_r/kernel (这个 kernel 其实就是内核镜像, 也就是 kernel/arch/arm64/boot/Image, 是拷贝过去重命名的)

--ramdisk out/target/product/rk3568_r/ramdisk.img (安卓生成的 ramdisk.img 镜像文件)

--dtb out/target/product/rk3568_r/dtb.img (内核源码的设备树镜像, 其实就是 kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtb)

--cmdline console=ttyFIQ0 androidboot.baseband=N/A androidboot.wificountrycode=CN
androidboot.veritymode=enforcing androidboot.hardware=rk30board androidboot.console=ttyFIQ0
androidboot.verifiedbootstate=orange firmware_class.path=/vendor/etc/firmware init=/init rootwait ro
loop.max_part=7 androidboot.selinux=permissive buildvariant=userdebug (这个是传递给内核的 cmdline,
这个不同板子自己去提供的, 譬如 RK 在 device/rockchip/common/BoardConfig.mk 或 BoardConfig_AB.mk
文件中提供了, 通过 BOARD_KERNEL_CMDLINE 定义的)

--os_version 11 (安卓系统版本)

--os_patch_level 2021-06-05 (补丁级别、什么鬼的)

--second kernel/resource.img (RK 的资源镜像 resource.img)

--header_version 2 (启动镜像 boot.img 头的版本)

(可以直接在安卓顶层源码目录下执行 make bootimage 去生成 boot.img)

这就是 boot.img 的制作过程了。所以安卓格式的 boot.img 包含了(内核镜像、内核设备树镜像、ramdisk、cmdline 以及 resource.img 资源镜像)。

制作得到的 boot.img 镜像文件, 有一个头部数据结构, 如下所示:

```
struct andr_img_hdr {  
    /* Must be ANDR_BOOT_MAGIC. */  
    char magic[ANDR_BOOT_MAGIC_SIZE];  
  
    u32 kernel_size; /* size in bytes */  
    u32 kernel_addr; /* physical load addr */  
  
    u32 ramdisk_size; /* size in bytes */  
    u32 ramdisk_addr; /* physical load addr */  
  
    u32 second_size; /* size in bytes */  
    u32 second_addr; /* physical load addr */  
  
    u32 tags_addr; /* physical addr for kernel tags */  
    u32 page_size; /* flash page size we assume */  
  
    /* Version of the boot image header. */  
    u32 header_version;  
  
    /* Operating system version and security patch level.  
    * For version "A.B.C" and patch level "Y- M- D":  
    * (7 bits for each of A, B, C; 7 bits for (Y- 2000), 4 bits for M)  
    * os_version = A[31:25] B[24:18] C[17:11] (Y- 2000)[10:4] M[3:0] */  
    u32 os_version;  
  
    char name[ANDR_BOOT_NAME_SIZE]; /* asciiz product name */  
  
    char cmdline[ANDR_BOOT_ARGS_SIZE];  
  
    u32 id[8]; /* timestamp / checksum / sha1 / etc */  
};
```



```

/* Supplemental command line data; kept here to maintain
 * binary compatibility with older versions of mkbootimg. */
char extra_cmdline[ANDR_BOOT_EXTRA_ARGS_SIZE];

/* Fields in boot_img_hdr_v1(Android- 9) and newer. */
u32 recovery_dtbo_size; /* size in bytes for recovery DTBO/ACPIO image */
u64 recovery_dtbo_offset; /* offset to recovery dtbo/acpio in boot image */
u32 header_size;

/* Fields in boot_img_hdr_v2(Android- 10) and newer. */
u32 dtb_size; /* size in bytes for DTB image */
u64 dtb_addr; /* physical load address for DTB image */

/*
 * [Rockchip compatibility]
 *
 * boot_img_hdr_v3(Android- 11) is not compatible with boot_img_hdr_v012,
 * we have to partly merge fields from boot_img_hdr_v3 and vendor_boot_img_hdr_v3
 * into this structure to compatible with boot_img_hdr_v012.
 */
u32 boot_ramdisk_size; /* size in bytes */
u32 vendor_ramdisk_size; /* size in bytes */
u32 vendor_page_size;
u32 vendor_header_version;
u32 vendor_header_size;
/*
 * Don't define 'char total_cmdline[TOTAL_BOOT_ARGS_SIZE]' to avoid
 * this structrue is over size than page_size.
 */
char *total_cmdline;
} __attribute__((packed));

```

通过 magic 可以校验镜像是不是安卓格式 boot.img

```
#define ANDR_BOOT_MAGIC "ANDROID!"
```

也就是比较 magic 是不是等于 "ANDROID!"

1.4 内核目录下单独编译内核镜像以及设备树镜像

单独编译内核镜像 Image:

```
make ARCH=arm64 Image -j24
```

单独编译设备树镜像:

```
make ARCH=arm64 rockchip/rk3568-alientek-evb1-v11.dtb -j24
```

1.5 单独编译生成 resource.img 镜像

首先在内核目录下生成 resource.img 镜像, 此时不包含充电显示相关的图片(这些图片都在 u-boot 目录下):

```
cd kernel && rm -rf resource.img && ./scripts/resource_tool \
arch/arm64/boot/dts/rockchip/rk3568-atk-evb1-ddr4-v10.dtb \
logo.bmp logo_kernel.bmp && cd -
```

然后再次生成 resource.img 镜像 去替换内核目录下的 resource.img 镜像文件:
进入到 u-boot 目录下去生成, 生成之后再去替换内核目录下的 resource.img。

```
cd u-boot && ./scripts/pack_resource.sh ../kernel/resource.img \  
&& cp resource.img ../kernel/resource.img && cd -
```

1.6 安卓目录下单独编译 boot.img 镜像

```
rm -rf out/target/product/rk3568_r/boot.img  
make bootimage
```

编译 boot.img 的时候会从内核源码目录下拿到 resource.img 镜像、kernel/arch/arm64/boot/Image 内核镜像以及 kernel/arch/arm64/boot/dts/rockchip/目录下的.dtb 镜像(根据后缀, 因为该目录下一般只有一个设备树镜像文件, make distclean 之后会删除该目录下生成的所有 dtb 镜像文件)。

```
单独编译 odm.img  
make odmimage
```

```
单独编译 product.img  
make productimage
```

```
单独编译 ramdisk.img  
make ramdisk
```

```
单独编译 recovery.img 镜像文件:  
make recoveryimage
```

```
单独编译 super.img  
make superimage
```

```
单独编译 system.img  
make systemimage
```

单独编译 vendor.img:

内核目录下的.ko 模块最终会进入到 vendor.img 镜像中, 也就是 vendor/lib/modules 这个目录下, 所以我们在编译 vendor.img 镜像的时候需要先编译内核、编译内核模块, 然后再编译 vendor.img 镜像, 在编译过程中会将内核中的 ko 模块一起放置到 vendor 镜像中:

```
make vendorimage
```